Sliding window

1.Fixed Size Window (Window size = k)

☞ Window length is already given.

Pattern

```
// create first window
// then slide by adding new element and removing old element
```

Questions

1. Maximum Sum Subarray of Size K

```
public:
  int maxSubarraySum(vector<int>& arr, int k) {
      // code here
      int n=arr.size();
      int sum=0;
      for(int i=0;i<k;i++){
          sum+=arr[i];
      }
      int maxsum=sum;
      for(int j=k;j<n;j++){

          sum+=arr[j];
          sum-=arr[j-k];
          maxsum=max(sum,maxsum);

      }
      return maxsum;
  }
};
```

2. Average of Subarrays of Size K

```
class Solution {
public:
    double findMaxAverage(vector<int>& arr, int k) {
        int n=arr.size();
        int sum=0;
        for(int i=0;i<k;i++){
            sum+=arr[i];
        }
        int maxsum=sum;
        for(int j=k;j<n;j++){

            sum+=arr[j];
            sum-=arr[j-k];
            maxsum=max(sum,maxsum);

        }
        return double(maxsum)/k;

```

3. Number of Negative Integers in Every Window of Size K

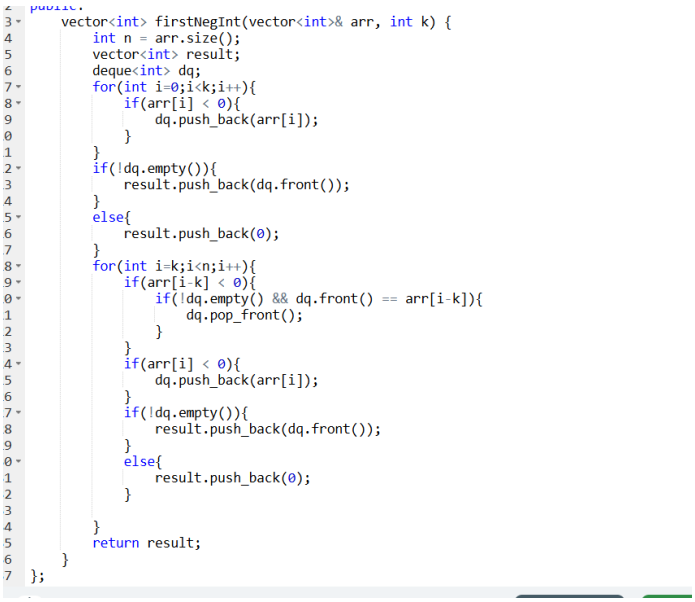
```
public:
    vector<int> firstNegInt(vector<int>& arr, int k) {
        int n = arr.size();
        vector<int> result;
        deque<int> dq;
        for(int i=0;i<k;i++){
            if(arr[i] < 0){
                dq.push_back(arr[i]);
            }
        }
        if(!dq.empty()){
            result.push_back(dq.front());
        }
        else{
            result.push_back(0);
        }
        for(int i=k;i<n;i++){
            if(arr[i-k] < 0){
                if(!dq.empty() && dq.front() == arr[i-k]){
                    dq.pop_front();
                }
            }
            if(arr[i] < 0){
                dq.push_back(arr[i]);
            }
            if(!dq.empty()){
                result.push_back(dq.front());
            }
            else{
                result.push_back(0);
            }

        }
        return result;
    }
};
```

4. Maximum in Every Window of Size K

```
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        multiset<int>m1;
        int n=nums.size();
        int j=0;
        vector<int>result;
        for(int i=0;i<n;i++){
            m1.insert(nums[i]);
            if(i-j+1==k){
                result.push_back(*m1.rbegin());
                m1.erase(m1.find(nums[j]));
                j++;
            }
        }
        return result;
    }
};
```

5. First Negative Number in Every Window of Size K



```
public:
    vector<int> firstNegInt(vector<int>& arr, int k) {
        int n = arr.size();
        vector<int> result;
        deque<int> dq;
        for(int i=0;i<k;i++){
            if(arr[i] < 0){
                dq.push_back(arr[i]);
            }
        }
        if(!dq.empty()){
            result.push_back(dq.front());
        }
        else{
            result.push_back(0);
        }
        for(int i=k;i<n;i++){
            if(arr[i-k] < 0){
                if(!dq.empty() && dq.front() == arr[i-k]){
                    dq.pop_front();
                }
            }
            if(arr[i] < 0){
                dq.push_back(arr[i]);
            }
            if(!dq.empty()){
                result.push_back(dq.front());
            }
            else{
                result.push_back(0);
            }

        }
        return result;
    }
};
```

2. Variable Size Window (Longest/Shortest)

☞ Window size is NOT fixed.

Pattern

Expand with `i`
Shrink with `j`

```
for(i=0;i<n;i++)
{
    // include arr[i]

    while(condition invalid)
    {
        // remove arr[j]
        j++;
    }

    ans=max(ans,i-j+1);
}
```

A. Longest Subarray/Substring

Questions

Easy

1. Longest Substring Without Repeating Characters

```
int lengthOfLongestSubstring(string s) {
    int n=s.length();
    unordered_set<int>mp;
    int j=0;
    int maxlen=INT_MIN;
    for(int i=0;i<n;i++){
        while(mp.count(s[i])){
         mp.erase(s[j]);
         j++;

        }
        mp.insert(s[i]);
        maxlen=max(maxlen,i-j+1);
    }
    return maxlen;
}
```

2. Max Consecutive Ones III

```
int longestOnes(vector<int>& nums, int k) {
    int n=nums.size();
    int j=0;
    int flip=0;
    int maxlen=0;
    for(int i=0;i<n;i++){
        if(nums[i]==0){
            flip++;
        }
        while(flip>k){
            if(nums[j]==0){
                flip--;
            }
            j++;
        }
        maxlen=max(maxlen,i-j+1);
    }
    return maxlen;
}
```

3. Longest Repeating Character Replacement

```
int characterReplacement(string s, int k) {
    int n=s.length();
    int j=0;
    int maxlen=0;
    int maxfreq=INT_MIN;
    vector<int>freq(26,0);
    for(int i=0;i<n;i++){
        freq[s[i]-'A']++;
        maxfreq=max(maxfreq,freq[s[i]-'A']);
        while((i-j+1)-maxfreq>k){
            freq[s[j]-'A']--;
            j++;
        }
     maxlen=max(maxlen,i-j+1);

    }
    return maxlen;
}
```

4. Fruit Into Baskets

```cpp
int totalFruit(vector<int>& fruits) {
    int n=fruits.size();
    int j=0;
    int maxlen=INT_MIN;
    unordered_map<int,int>mp;
    for(int i=0;i<n;i++){
        mp[fruits[i]]++;
        while(mp.size()>2){
            mp[fruits[j]]--;
            if(mp[fruits[j]]==0){
                mp.erase(fruits[j]);
            }
            j++;
        }
        maxlen=max(maxlen,i-j+1);
    }
    return maxlen;

}
```

5. Longest Substring with At Most K Distinct Characters

```cpp
public:
    int kDistinctChar(string& s, int k) {
        int n=s.length();
        int maxlen=0;
        int j=0;
        unordered_map<char,int>mp;
        for(int i=0;i<n;i++){
            mp[s[i]]++;
            while(mp.size()>k){
                mp[s[j]]--;
                if(mp[s[j]]==0){
                    mp.erase(s[j]);
                }
                j++;
            }
            maxlen=max(maxlen,i-j+1);
        }
        return maxlen;
    }
};
```

Medium

6. Longest Nice Subarray
7. Longest Subarray of 1's After Deleting One Element
8. Get Equal Substrings Within Budget

B. Smallest/Minimum Window

Pattern

```
while(valid)
{
    ans=min(ans,i-j+1);
    shrink();
}
```

Questions

1. Minimum Size Subarray Sum

```
public:
    int minSubArrayLen(int target, vector<int>& nums) {
        int n=nums.size();
        int sum=0;
        int minlen=INT_MAX;
        int j=0;
        for(int i=0;i<n;i++){
            sum+=nums[i];
            while(sum>=target){
                minlen=min(minlen,i-j+1);
                sum-=nums[j];
                j++;
            }
        }
        return (minlen==INT_MAX)?0:minlen;
```

2. Minimum Window Substring
3. Smallest Subarray with Sum ≥ K

3.Count Subarrays Pattern

Most important interview pattern.

Pattern

Find:

```
count(atMost(k))
```
Then
```
exactly(k)=atMost(k)-atMost(k-1)
```

Questions

1. Subarrays with K Different Integers

```
public:
int atmostk(vector<int>& nums,int k){
    int n=nums.size();
    int count=0;
    unordered_map<int,int>mp;
    int j=0;
    for(int i=0;i<n;i++){
        mp[nums[i]]++;
        while(mp.size()>k){
            mp[nums[j]]--;
            if(mp[nums[j]]==0){
                mp.erase(nums[j]);
            }
            j++;
        }
        count+=i-j+1;
    }
    return count;
}
    int subarraysWithKDistinct(vector<int>& nums, int k) {
        return atmostk(nums,k)-atmostk(nums,k-1);

    }
};
```

2. Count Number of Nice Subarrays

```
public:
int atmost(vector<int>& nums,int k){
    int n=nums.size();
      int j=0;
      int count=0;
      for(int i=0;i<n;i++){
          if(nums[i]%2==1){
              k--;
          }
              while(k<0){
                  if(nums[j]%2==1){
                      k++;
                  }
                  j++;
              }
              count+=i-j+1;
          }
       return count;
}
    int numberOfSubarrays(vector<int>& nums, int k) {
      return atmost(nums,k)-atmost(nums,k-1);

    }
};
```

3. Binary Subarrays With Sum

```
public:
int atMost(vector<int>& nums, int k){
    if(k<0) return 0;
    int n=nums.size();
    int count=0;
    int j=0;
    int sum=0;
    for(int i=0;i<n;i++){
        sum+=nums[i];
        while(sum>k){
            sum-=nums[j];
            j++;
        }
        count+=i-j+1;
    }
    return count;
    }
    int numSubarraysWithSum(vector<int>& nums, int goal) {
        return atMost(nums,goal)-atMost(nums,goal-1);
    }
};
```

4. Count Vowel Substrings

```
public:
bool isvowel(char c){
    return c=='a'||c=='e'||c=='i'||c=='o'||c=='u';
}
    int countVowelSubstrings(string word) {
        int n=word.size();
        int count=0;
        for(int i=0;i<n;i++){
        unordered_map<char,int>mp;
            for(int j=i;j<n;j++){
                if(!isvowel(word[j])){
                    break;
                }
                mp[word[j]]++;

            if(mp.size()==5) count++;
            }
        }
        return count;
    }
};
```

5. Number of Substrings Containing All Three Characters

```
public:
    int numberOfSubstrings(string s) {
       vector<int>freq(26,0);
       int n=s.length();
       int l=0;
       int count=0;
       for(int i=0;i<n;i++){
        freq[s[i]-'a']++;
        while(freq[0]>0&&freq[1]>0&&freq[2]>0){
            freq[s[l]-'a']--;
            l++;
        }
        count+=l;
       }
        return count;
    }
};
```

4.Sliding Window + Frequency Array

Use:

```
vector<int> freq(26,0);
```

Questions:

1. Permutation in String

```
public:
    bool checkInclusion(string s1, string s2) {
        int n1=s1.size();
        int n2=s2.size();
        if(n1>n2) return false;
        vector<int>freq1(26,0);
        vector<int>freq2(26,0);

        for(auto x:s1){
            freq1[x-'a']++;
        }
       for(int i=0;i<n1;i++){
        freq2[s2[i]-'a']++;
       }
       if(freq1==freq2) return true;

       for(int i=n1;i<n2;i++){
        freq2[s2[i]-'a']++;
        freq2[s2[i-n1]-'a']--;
        if(freq1==freq2) return true;
       }
       return false;


    }
};
```

2. Find All Anagrams in a String

```
public:
    vector<int> findAnagrams(string s, string p) {
        vector<int>v1(26,0);
        vector<int>v2(26,0);
        vector<int>ans;
       int n1=s.length();
       int n2=p.length();
       if(n1<n2) return ans;
       for(int i=0;i<n2;i++){
          v2[p[i]-'a']++;
        }
        for(int i=0;i<n2;i++){
          v1[s[i]-'a']++;
        }
        if(v1==v2){
          ans.push_back(0);
        }
        for(int i=n2;i<n1;i++){
          v1[s[i]-'a']++;
          v1[s[i-n2]-'a']--;
          if(v1==v2){
              ans.push_back(i-n2+1);
          }
        }
        return ans;

    }
};
```

5. Sliding Window + Deque

Used when you need **maximum/minimum in current window**.

Questions

1. Sliding Window Maximum ⭐

```
vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    deque<int>dq;
    vector<int>ans;
    int n=nums.size();
    for(int i=0;i<n;i++){
        while(!dq.empty()&&nums[dq.back()]<nums[i]){
            dq.pop_back();
        }
        dq.push_back(i);
         while(!dq.empty() && dq.front() <= i-k){
    dq.pop_front();
         }
        if(i>=k-1){
             ans.push_back(nums[dq.front()]);

        }
    }
    return ans;
}
;
```

2. Sliding Window Minimum –same as previous
3. Shortest Subarray with Sum at Least K

Pattern:

```
C++

deque<int> dq;

for(int i=0;i<n;i++){

    while(!dq.empty() && condition)
        dq.pop_back();

    dq.push_back(i);

    while(!dq.empty() && outdated_index)
        dq.pop_front();

    // answer from dq.front()
}
```

The only thing that changes is the **condition**.

Recommended LeetCode Order

Beginner

1. Maximum Average Subarray I
2. Max Consecutive Ones III
3. Longest Substring Without Repeating Characters
4. Fruit Into Baskets

Intermediate

5. Longest Repeating Character Replacement
6. Permutation in String
7. Find All Anagrams in a String
8. Minimum Size Subarray Sum
9. Sliding Window Maximum

Advanced

10. Binary Subarrays With Sum
11. Count Number of Nice Subarrays
12. Subarrays with K Different Integers
13. Minimum Window Substring

For placements, if you master these **10 questions**:

- Longest Substring Without Repeating Characters -done
- Fruit Into Baskets -done
- Longest Repeating Character Replacement -done
- Permutation in String -done
- Find All Anagrams in a String -done
- Minimum Size Subarray Sum
- Sliding Window Maximum -done
- Binary Subarrays With Sum -done
- Subarrays with K Different Integers -done
- Minimum Window Substring –not done